CSE 144 Final Project Report

Transfer Learning Challenge

Contributors:

Aashna Shimkhada (CruzID: aashimkh)

Spring 2026

1 Introduction

Problem goal and setting. This project addresses a 100-class image classification challenge on Kaggle for CSE 144. The goal is to predict the category label (0–99) for each of 1,000 unlabeled test images using only ~10 labeled training images per class.

Why transfer learning is appropriate. With only ~8 effective training images per class (after a 15% val split), a randomly initialized network memorizes the training set within a few epochs and fails to generalize. Transfer learning from EfficientNet-B3 pretrained on ImageNet (1.2M images, 1,000 classes) provides general visual representations that need only minor adaptation, bypassing the data scarcity problem.

Summary of approach and main result. EfficientNet-B3 is fine-tuned using a two-phase strategy with differential learning rates and gradual block unfreezing. The pipeline was developed iteratively over four runs, diagnosing and correcting both overfitting (v1) and underfitting (v2/v3) before reaching a converged solution (v4). A bug in the inference pipeline (ID format mismatch in the TTA submission builder) was identified and corrected in the final version. Final best validation accuracy: **~61%** | Kaggle public leaderboard: **60%**

2 Dataset

Number of classes and dataset sizes. 100 classes, ~10 training images per class (1,000 total training images). A 15% validation split (seeded) yields ~850 train / ~150 val images. The test set contains 1,000 unlabeled images.

Directory structure and label mapping. The `train/` directory contains 100 subdirectories named `"0"` through `"99"`. PyTorch's `ImageFolder` sorts folder names lexicographically, so `"10"` would sort before `"2"`, silently scrambling all label assignments. The fix: zero-pad all single-digit folder names

(`"0"` → `"00"`, ..., `"9"` → `"09"`) before loading, so alphabetical and numerical orders agree. The notebook verifies `class_to_idx["00"] == 0` for all 100 folders with an assertion before training begins. The `test/` directory contains `0.jpg`–`999.jpg`, sorted by integer stem at inference time.

Preprocessing and augmentation. All images resized to 224×224 and normalized with ImageNet statistics (mean `[0.485, 0.456, 0.406]`, std `[0.229, 0.224, 0.225]`). Training augmentations: random crop from 256×256, horizontal flip (p=0.5), vertical flip (p=0.1), rotation (±15°), color jitter (brightness/contrast/saturation ±0.3, hue ±0.1), random grayscale (p=0.05), Gaussian blur (σ=0.1–2.0), random erasing (p=0.3). Mixup (α=0.4) applied during Phase 2 at the batch level. Label smoothing (ε=0.1) applied at the loss level. Validation and test: resize and normalize only.

3 Implementation

3.1 Model

Pretrained backbone. EfficientNet-B3 pretrained on ImageNet (`EfficientNet_B3_Weights.DEFAULT`, ~12M parameters). Chosen over B2 for its slightly richer feature space (1,536 vs. 1,408 head input features), with still-manageable parameter count for the ~850-sample training set. ResNet-50 (~25M) and ViT-Small (~22M) overfit more severely in preliminary experiments.

Architecture changes. Original 1,000-class classifier replaced with:

Dropout(0.4) → Linear(1536 → 512) → ReLU → Dropout(0.3) → Linear(512 → 100)

Fine-tuning strategy. Two phases with gradual unfreezing:

- **Phase 1 (10 epochs):** Backbone fully frozen. Only the classifier head trains at LR=3e-3. No Mixup. Stabilizes the randomly initialized head before exposing backbone to gradients.
- **Phase 2 (up to 40 epochs, early stopping):** Top 4 backbone blocks unfrozen at epoch 1; top 6 blocks at epoch 11. Differential LRs: backbone at 1e-4 (10× slower than head at 1e-3). Newly unfrozen blocks at epoch 11 receive 5e-5 (0.5× backbone rate). Early stopping with patience=12 saves the best checkpoint.

3.2 Training

Loss, optimizer, scheduler. Cross-entropy with label smoothing (ε=0.1). AdamW with weight decay 1e-4. Gradient clipping (max norm=1.0). CosineAnnealingWarmRestarts (T_0=10); periodic LR restarts help escape sharp local minima.

Final hyperparameters (v4):

Parameter	Phase 1	Phase 2
Epochs	10	≤40 (early stop)
Head LR	3e-3	1e-3
Backbone LR	–	1e-4
Newly unfrozen blocks LR	–	5e-5
Weight decay	1e-4	1e-4
Batch size	32	32
Mixup α	–	0.4
Label smoothing	0.1	0.1
Grad clip norm	1.0	1.0
Early stop patience	–	12

Hardware. Google Colab T4 GPU. Data copied from Google Drive to Colab's local SSD at runtime (eliminates Drive I/O bottleneck, ~10× faster DataLoader throughput). Estimated total training time: ~18–20 minutes.

4 Experiments

The four-run experimental progression:

v1 (CPU, 40 ep, no Mixup, single LR): Train accuracy converged to ~100%, val accuracy plateaued at ~60%, with a ~40-point gap that never closed. Val loss was flat while train loss continued falling, classic memorization. Phase 2 (all layers, single LR=3e-4) provided zero val improvement.

v2 (CPU, 15 ep, Mixup α=0.4, LR=5e-5): Over-regularized. Train acc (~25%) fell below val acc (~42%), a Mixup artifact where training accuracy is measured on blended images (harder than clean), artificially deflating the number. Val loss still declining at epoch 15 meaning the model had not converged; training was stopped too early. Also: running on CPU caused super long run times.

v3 (GPU, 35 ep, Mixup, differential LRs introduced): Val acc reached ~57%, trending upward at epoch 35 with val loss still declining. Model not yet converged at cutoff.

v4 (GPU, 50 ep, full pipeline): Val accuracy converged to ~61%, plateau evident from epoch ~40. Train accuracy noisy from Mixup but tracking val. Val loss slowly declining, a few more epochs could extract 1–2% more.

Bug discovered and fixed (v4): The `TestDataset` in the TTA inference pipeline returned `p.stem` (e.g., `"42"`) as the image ID. The `sample_submission.csv` uses `"42.jpg"` as the ID. The `.map()` found no key matches and silently produced all-NaN labels explaining why the uploaded submission CSV had empty labels. Fixed by returning `p.name` instead, with an assertion that zero null labels remain after mapping.

Ablation summary:

Variant	Val Acc	Key issue
v1: baseline, CPU, 40 ep	~60%	Severe overfitting, train→100%
v2: Mixup, LR=5e-5, CPU, 15 ep	~42%	Under-regularized schedule, not converged
v3: GPU, 35 ep, diff LRs	~57%	Truncated; still improving
v4: GPU, 50 ep, bug fixed	~61%	Converged, above baseline

5 Results

Training and validation curves. Phase 1 (frozen backbone) brings val acc to ~33% within 10 epochs. Phase 2 (gradual unfreeze) pushes val to ~61% with early stopping triggered around epoch 40–50. In Phase 2, train acc appears lower than val acc on the plot, this is the expected Mixup artifact, not a sign of underfitting; val on clean images is the true metric.

Kaggle public leaderboard score: 60%

TTA. Three augmentation views (original, center-crop, horizontal flip) are averaged at inference. This provides a consistent ~1% improvement over single-pass inference at no training cost.

Error analysis. The model performs best on visually distinctive classes (unique textures, silhouettes) and struggles most on fine-grained categories with high intra-class visual diversity. Some errors arise from domain shift, the 100 classes are sampled from multiple source datasets, so ImageNet features don't transfer equally to all.

6 Discussion

What worked best and why. The most impactful single change was switching from a single learning rate for all parameters to differential LRs (backbone 10× slower than head). This allowed Phase 2 to meaningfully adapt the backbone's higher-level features without disrupting the well-calibrated lower-level ones. Gradual block unfreezing reinforced this by adapting task-specific top blocks first before touching general early-layer features.

Mixup was effective at preventing the memorization seen in v1, though α=0.4 makes the train accuracy metric unreliable as a diagnostic (the Mixup artifact). An α of 0.2 would retain regularization while producing more readable curves.

Failure cases. Some fine-grained class pairs with similar visual appearance consistently confuse the model. A prototype-based classifier (nearest centroid) might handle these better than a linear layer, since it doesn't need to learn a decision boundary, it just measures distance to the class mean in feature space.

Limitations and next steps.

1. CutMix: blends image patches; generally outperforms pixel-level Mixup
2. Ensemble: 3 models × different seeds, typically +2–3%
3. Full backbone unfreeze: currently only top 6 of 9 blocks; all layers at very low LR may gain more
4. Self-supervised backbone (DINO/MAE): better cross-domain transfer than supervised ImageNet
5. More epochs: val loss still declining at epoch 50; convergence not fully reached

7 Reproducibility

Seeds and determinism. Global seed 42 for Python `random`, NumPy, and PyTorch (CPU and CUDA). `cudnn.deterministic=True`, `benchmark=False`. Train/val split uses a seeded `torch.Generator`. With the same seed and T4 GPU, val accuracy should reproduce within ±1%.

Environment: Google Colab T4 GPU. Python 3.10, PyTorch 2.x+cu118, TorchVision 0.x+cu118. All packages pre-installed on Colab.

Commands to train and generate submission:

bash

```
# Open cse144_transfer_learning.ipynb in Google Colab
# Set runtime: Runtime → Change runtime type → T4 GPU
# Run All cells (Runtime → Run all)
# Cell 13 auto-downloads submission.csv and saves best_model.pth to Drive
```

8 Team Contributions

Solo project by Aashna:

- Data pipeline, label-mapping fix (folder zero-padding + assertion)
- Model architecture (EfficientNet-B3 + custom head)
- Iterative training: v1 overfitting → v2 over-regularization → v3 truncated → v4 converged
- Differential LR and gradual block unfreezing implementation
- TTA inference bug discovery and fix (`p.stem` → `p.name`)
- Report, README, presentation, Q&A prep

9 References

1. TorchVision pretrained models: https://pytorch.org/vision/stable/models.html
2. Tan, M. & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for CNNs. *ICML 2019*.
3. Zhang, H. et al. (2018). Mixup: Beyond Empirical Risk Minimization. *ICLR 2018*.
4. Loshchilov, I. & Hutter, F. (2019). Decoupled Weight Decay Regularization. *ICLR 2019*.
5. Müller, R. et al. (2019). When Does Label Smoothing Help? *NeurIPS 2019*.
6. PyTorch documentation: https://pytorch.org/docs/stable/